

Measurement Technics  
Project Report  
Group 43 - KuDAQ

Authors Names

May 23, 2026

## Contents

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                               | <b>3</b>  |
| <b>2</b> | <b>Measurements</b>                               | <b>4</b>  |
| 2.1      | Measurement 1: Orientation . . . . .              | 4         |
| 2.1.1    | Objective . . . . .                               | 4         |
| 2.1.2    | Equipment and Setup . . . . .                     | 4         |
| 2.1.3    | Procedure . . . . .                               | 9         |
| 2.1.4    | Results . . . . .                                 | 9         |
| 2.1.5    | Discussion . . . . .                              | 14        |
| 2.1.6    | Conclusions . . . . .                             | 15        |
| <b>3</b> | <b>Additional Measurements</b>                    | <b>16</b> |
| 3.1      | Measurement 2: [Title to be determined] . . . . . | 16        |
| <b>4</b> | <b>Conclusion</b>                                 | <b>17</b> |
| <b>A</b> | <b>Raw Data</b>                                   | <b>19</b> |
| <b>B</b> | <b>Software and Configuration</b>                 | <b>19</b> |



## 2 Measurements

### 2.1 Measurement 1: Orientation

#### 2.1.1 Objective

The objective of the first measurement is to evaluate the impact of some environmental parameters on the movement of a standardized object caught in a jet flow. In order to do this, we will use the orientation of the object v.s. time, for a given set of parameters, during a fixed time interval. The jet flow is to be generated by a water pump in a water tank of approximately 5L capacity. The standardized object must be small enough to be free to move around the water tank, must not be physically linked to the acquisition system to remain fully independent to avoid potential biases, and must be heavy enough to be able to be caught in the jet flow, but not too heavy to be able to move around the water tank. The parameters that we were able to identify as potentially impacting the movement of the object are the following:

- The nozzle shape and size of the water pump
- The shape of the object
- Some other objects placed in the water tank
- The position of the water pump in the water tank
- The water level in the water tank

We eventually decided for this experiment to focus on the position of the water pump in the water tank, as it is a simple parameter to modify between tests, and that we expect to have a significant impact on the movement of the object. We planned to focus the measurement on the height of the water pump in the water tank, keeping the exact same horizontal position. Because the object will be floating on the water surface, we expect to find larger movement amplitude when the water pump is closer to the water surface, and smaller movement amplitude when the water pump is further from the water surface.

#### 2.1.2 Equipment and Setup

##### Electrical Hardware

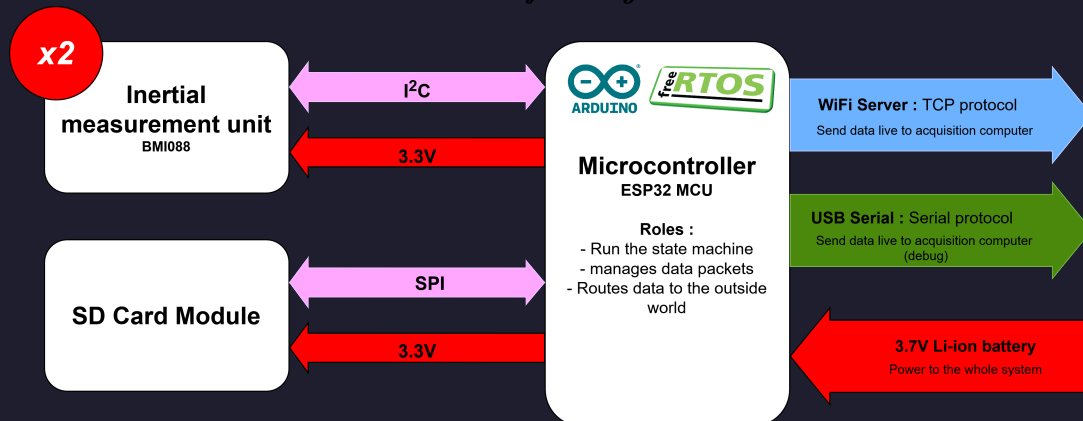
For this measurement, we needed to extract the orientation of a small object versus time in an isolated experimental setup. To do this, we designed a fully integrated custom data acquisition system. The main requirements of this system were :

- To be able to acquire data from an inertial measurement unit (IMU) sensor, in order to extract the orientation of the object versus time
- To be able to acquire data at a high sampling frequency, in order to capture the movement of the object with a good resolution (i.e. at least 50Hz)
- To be able to store internally, or to stream in real-time the measured data, in order to perform data analysis later on.
- To be able to operate in a water environment, as the experiment is taking place in a water tank
- To be fully independent and self-contained, in order to avoid any potential bias on the measurement

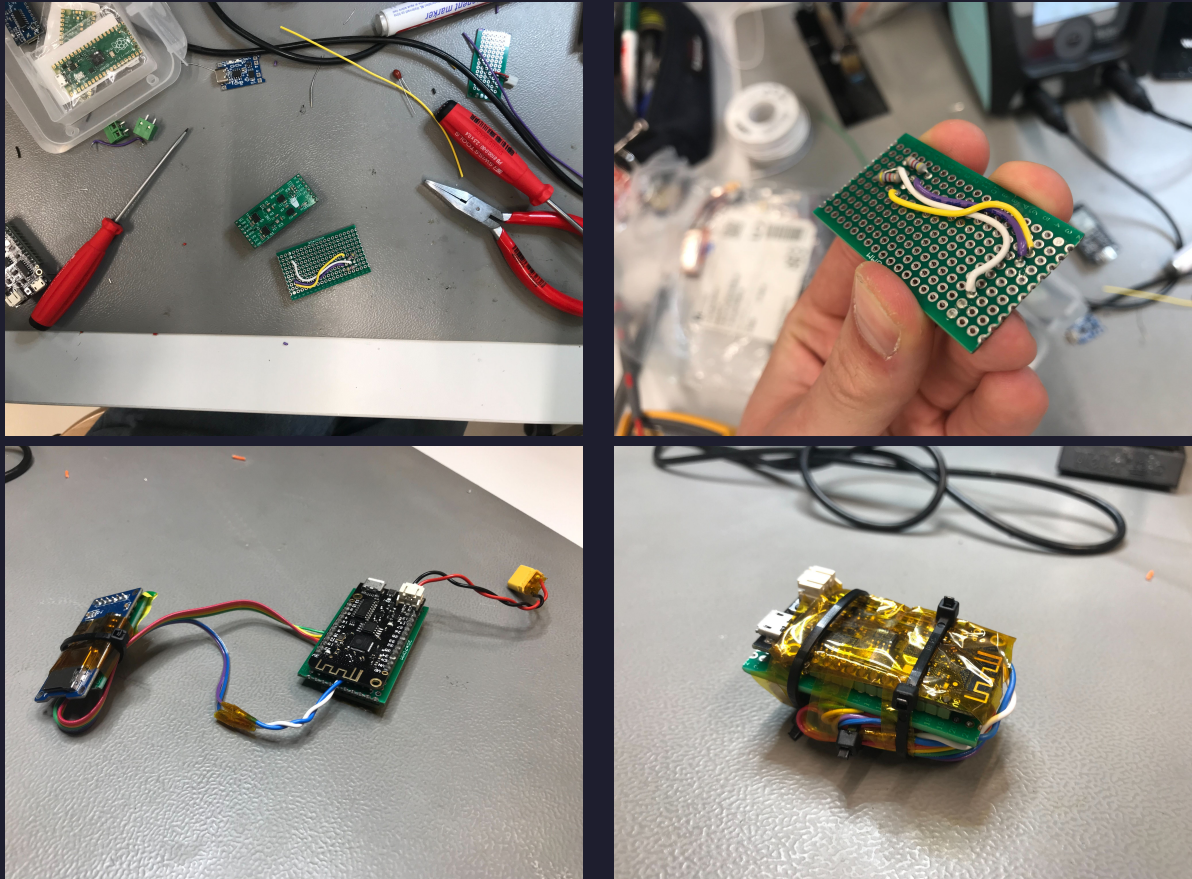
To meet these requirements, we designed the KuDAQ system, which is a custom data acquisition system based on a microcontroller and an IMU sensor, which is able to record acceleration and angular velocity data on all 3 axis. The microcontroller used is a small low-power development board based on the Espressif ESP32 chip, which is able to operate at high frequency, has multiple communication interfaces, has 2 CPU cores, and has bluetooth and WiFi connectivity. The IMU sensor used is the Bosch Sensortec BMI088, which is a high-performance 6-axis IMU sensor, with a wide measurement range and a high sampling frequency. The system is completed by an SD card read & write module, which allows to store the measured data internally if needed (this feature was eventually not implemented in the software, even if the board was capable of doing it). The power management of the system is entirely handled by the dev board, as it has a built-in battery management system, which allows to power the system with a rechargeable battery, and to charge it through a USB connection.

In order to communicate with the devices, the microcontroller uses I2C bus for the BMI088 sensor, and SPI bus for the SD card module. USB-Serial communication is used to stream data to a wired computer for debugging purposes, finally, WiFi communication through TCP/IP protocol is used to stream data in real-time wirelessly to a computer during the experiment, to avoid interfering with the device by having a cable wired in.

*The overall schematic of the system looks like this:*



**Figure 1:** KuDAQ system schematic.



**Figure 2:** Assembly and manufacturing photos of the KuDAQ board.

Below are several pictures taken during the manufacturing of the board. We opted for a compact hand soldered board stack to minimise the volume taken by the system

### Mechanical Hardware

The mechanical hardware of this system is relatively simple, as it only needs to act as a hollow shell to protect the electronic components from water, and to allow the object to float on the water surface. We first designed and 3d printed a hollow sphere in 2 parts. After initial testing of just the sphere, we found that the sphere was too smooth as an object to be caught and moved by the jet flow. We then decided to redesigned the object as a hollow square, with thick walls and large infill to ensure water tightness. The 3D print was then coated with a layer of transparent acrylic to furthermore ensure leaktightness of the PETG material. After initial testing of the square, we found that the object was correctly caught in the jet flow, but wasn't correctly leaktight, as water was penetrating the object through the joining surface between the two shells. To solve this problem, we 3D printed a custom TPU gasket to be placed between the shells. After extensive testing in the water immersed for several minutes, the object was finally fully leaktight and able to float on the water surface without any water penetration. for even more protection, we decided that during the real experiment, we would place the electronic board inside a closed latex glove, to ensure that even if some small amount of liquid was penetrating the object, it would not damage the circuit.

## Software Architecture

The firmware is written in C++ for the ESP32 microcontroller and is organised to separate sensing, signal processing, and communication using FreeRTOS and modular design principles.

### *FreeRTOS implementation diagram:*

– insert diagram here –

The program creates two concurrently running tasks pinned to different CPU cores :

- **Acquisition Task:** responsible for reading raw sensor data from the BMI088 at a fixed sampling rate, applying low-pass filtering to reduce noise, and computing orientation estimates using an AHRS algorithm.
- **Communication Task:** responsible for handling TCP/IP connections, parsing incoming commands to adjust parameters (e.g., sampling rate, filter settings), and streaming the latest sensor data to the acquisition computer in real-time.

In order to meet the requirements of high sampling frequency, low latency, and reliable data streaming in a potentially noisy environment, several key design choices were made in the software architecture, including:

- **Dual-core task pinning:** acquisition and communication run on separate CPU cores. This isolates time-critical sampling from the variable execution time of communication tasks, ensuring that sensor reads occur at constant intervals.

```
void setup() {
    DAQ.initialize_all(); // Initialize all components of the system
    xTaskCreatePinnedToCore(taskCore0, "Task0", 10000, NULL, 1, &Task1, 1); // Core 1
    xTaskCreatePinnedToCore(taskCore1, "Task1", 10000, NULL, 1, &Task2, 0); // Core 0
}
```

- **Delay-free acquisition loop:** the acquisition task does not use blocking waits, instead it checks elapsed time using the millis() function and performs reads only when the sampling interval has passed. This approach enables deterministic timing for sensor reads and helps maintaining a stable sampling frequency.

```
bool Sensor::read_data() {
    // define time variable
    static unsigned long last_timestamp = 0;
    // read both accel and gyro via the bmi wrapper
    bmi->readSensor();
    // check if enough time has passed since the last reading based on the sampling rate
    unsigned long intervalMs = (unsigned long)(1000.0f / sampling_rate);
    if (millis() - last_timestamp >= intervalMs) {
        #ifdef DEBUG
            Serial.println("Reading new data from sensor...");
        #endif
        last_timestamp = millis();
    }
}
```

- **Bounded, single-element queues with overwrite :** freshly acquired and processed sensor data is sent to the communication thread via a FreeRTOS queue with only one sample element, ensuring that only the most recent data packet is sent to the acquisition computer.

```
case CORE0_ORIENT_ACQ:
    // data acquisition and processing
    sensor1->acquisition();
    orientation1_sender = sensor1->getRawOrientation();
    // send only if there's a new data
    if (orientation1_sender.newData) {
        // For now, send orientation data over the queue
        xQueueOverwrite(orientationQueue, &orientation1_sender);
        orientation1_sender.newData = false; // Mark data as sent
    }
```

- **Circular sample buffers:** The sensor data is stored in a circular buffer in the acquisition task, allowing for efficient storage of a known number of samples for filtering purposes (potentially useful for moving average filters, not implemented as of now).

```
// Move data in the circular buffer to make place for the new data
raw_accel_data.index = (raw_accel_data.index + 1) % 10; // circular buffer index
raw_gyro_data.index = (raw_gyro_data.index + 1) % 10; // circular buffer index
// Add new data at the last position
raw_accel_data.x[raw_accel_data.index] = bmi->getAccelX_maa();
raw_accel_data.y[raw_accel_data.index] = bmi->getAccelY_maa();
raw_accel_data.z[raw_accel_data.index] = bmi->getAccelZ_maa();
raw_gyro_data.x[raw_gyro_data.index] = bmi->getGyroX_rads();
raw_gyro_data.y[raw_gyro_data.index] = bmi->getGyroY_rads();
raw_gyro_data.z[raw_gyro_data.index] = bmi->getGyroZ_rads();
raw_accel_data.timestamp[raw_accel_data.index] = last_timestamp;
raw_gyro_data.timestamp[raw_gyro_data.index] = last_timestamp;
// increment the unacknowledged new data index
raw_accel_data.unacked_data++;
raw_gyro_data.unacked_data++;
// overflow protection
if (raw_accel_data.unacked_data > 10) {
    raw_accel_data.unacked_data = 10; // cap at buffer size
}
if (raw_gyro_data.unacked_data > 10) {
    raw_gyro_data.unacked_data = 10; // cap at buffer size
}
```

- **Configurable IIR filtering:** first and second-order low-pass filters are implemented with coefficients computed from the sampling rate and cutoff frequency. The filter's parameters can be adjusted at runtime via commands.

```
void Sensor::filter_data_1stOrd() {
    #ifdef DEBUG
        Serial.println("Filtering data with 1st order filter...");
    #endif
    // Move all value by 1 position in the filtered buffers to make place for the new data
    filtered_accel_data.index = (filtered_accel_data.index + 1) % 10; // circular buffer index
    filtered_gyro_data.index = (filtered_gyro_data.index + 1) % 10; // circular buffer index
    // apply the first order filter to the new data and store in the filtered buffers
    if (filtered_accel_data.index == 0) {
        filtered_accel_data.x[filtered_accel_data.index] = f1st0ndev_LP(raw_accel_data.x[raw_accel_data.index]);
        filtered_accel_data.y[filtered_accel_data.index] = f1st0ndev_LP(raw_accel_data.y[raw_accel_data.index]);
        filtered_accel_data.z[filtered_accel_data.index] = f1st0ndev_LP(raw_accel_data.z[raw_accel_data.index]);
        filtered_gyro_data.x[filtered_gyro_data.index] = f1st0ndev_LP(raw_gyro_data.x[raw_gyro_data.index]);
        filtered_gyro_data.y[filtered_gyro_data.index] = f1st0ndev_LP(raw_gyro_data.y[raw_gyro_data.index]);
        filtered_gyro_data.z[filtered_gyro_data.index] = f1st0ndev_LP(raw_gyro_data.z[raw_gyro_data.index]);
    }
}
```

- **On-device orientation fusion:** a lightweight AHRS filter (Madgwick type algorithm) fuses accelerometer and gyroscope data into roll/pitch/yaw. Producing fused orientation on-device reduces the amount of data to stream and provides immediately usable orientation data for real-time applications.

```
void Sensor::AHRS_update() {
    // Move all value by 1 position in the orientation buffers to make place for the new data
    raw_orientation_data.index = (raw_orientation_data.index + 1) % 10; // circular buffer index
    // apply the AHRS filter to the new data and store in the orientation buffers
    ahrs->updateIMU(raw_gyro_data.x[raw_gyro_data.index], raw_gyro_data.y[raw_gyro_data.index], raw_gyro_data.z[raw_gyro_data.index], raw_accel_data.x[raw_accel_data.index], raw_accel_data.y[raw_accel_data.index], raw_accel_data.z[raw_accel_data.index]);
    raw_orientation_data.roll[raw_orientation_data.index] = ahrs->getRoll();
    raw_orientation_data.pitch[raw_orientation_data.index] = ahrs->getPitch();
    raw_orientation_data.yaw[raw_orientation_data.index] = ahrs->getYaw();
    raw_orientation_data.timestamp[raw_orientation_data.index] = (long)(raw_accel_data.timestamp[raw_accel_data.index]);
    raw_orientation_data.unacked_data++;
    if (raw_orientation_data.unacked_data > 10) {
        raw_orientation_data.unacked_data = 10; // cap at buffer size
    }
}
```

- **Non-blocking command handling and simple TCP control:** the communication task listens for incoming TCP connections and parses simple text-based commands to adjust parameters or request data. This allows for dynamic control of the acquisition process without interrupting the sampling loop.

```
void KuDAQ::WiFi_setup() {
    // WiFi connection setup
    WiFi.softAP(ssid, password); // Start WiFi in Access Point Mode with given SSID and password
    Serial.println("Access Point started!");
    Serial.print("IP address: ");
    Serial.println(WiFi.softAPIP()); // usually 192.168.4.1
    server.begin(); // Start the TCP server on port 23
    delay(100); // wait for WiFi to initialize
}
```

These choices prioritise timely, predictable sampling and low-latency delivery of recent measurements, which are the primary requirements for a measurement system intended to capture dynamic motion in a fluid environment.

## Acquisition Method

In order to monitor the orientation of the object live during the test, we developed a custom plotting software using Processing, which is able to receive the data streamed by the KuDAQ system through WiFi, and to show a real-time visualization of a cube representing the object, which is moving according to the received data. This software can also trigger the recording of the data in a .log file, specially encoded to be easily processed later with matlab for more advanced data analysis.

The acquisition procedure is the following :

- The KuDAQ system is turned on by plugging the battery, and is placed in the sealed container.
- The acquisition laptop is connected to the KuDAQ WiFi network.
- The acquisition software is launched
- The user starts the stream by clicking on the "Start Stream" button in the acquisition software, which sends a command to the KuDAQ system to start streaming data, and starts showing the real-time movement of the object on the screen.
- The user can then click on the "Start Recording" button in the acquisition software, which starts recording the received data in a .log file.
- When the battery runs low (approximately 1 hour of operation), the system turns off.

### 2.1.3 Procedure

Since the movement of the object is really slow, we decided to perform a total of 45 minutes of acquisition for each environmental condition, in order to have a good amount of data to analyse later on. During the acquisition, we made sure to not touch the water tank, and to avoid any contact with the object, in order to avoid any potential bias on the measurement. We decided to perform 3 acquisitions with the water pump at different heights in the water tank, while keeping the same horizontal position. The heights were defined to be :

- 5 cm from the water surface
- 2.5 cm from the water surface
- 1 cm from the water surface

In fact, we observed that placing the pump deeper than 5cm from the water surface was not creating a strong enough water flow to move the object around the tank.

### 2.1.4 Results

#### Visual observations :

During the test, we could clearly observe that the object was turning around itself, around the vertical axis, at a rhythm of about 1 turn every 10 seconds. After a few second after dropping the object, it quickly achieved a pseudo steady state, where the movement of the object was relatively stable. Nonetheless, we observed that the movement was heavily impacted by repeated contact between the cube and tank walls.

#### Data Analysis :

The plots below were directly extracted from the raw data streamed by the KuDAQ system during the experiment. By zooming on the graph, we can clearly observe the periodic movement of the object. coherent with the visual observations, we can see that mainly one axis (the yaw axis) is moving with great amplitude compared to the other 2. The yaw axis orientation data is indexed in degrees between 0 and 360, and we can see that the angle is increasing in a linear fashion from 0 to 360, and then effectively dropping back to 0, which is coherent with the fact that the object is turning around itself in a consistent direction. The roll and pitch axis are

relatively stable around 0, with some small amplitude noise, which is coherent with the fact that the object is not really tilting on the water surface, but rather just turning around itself.



**Figure 3:** Orientation of the object versus time during the experiment.

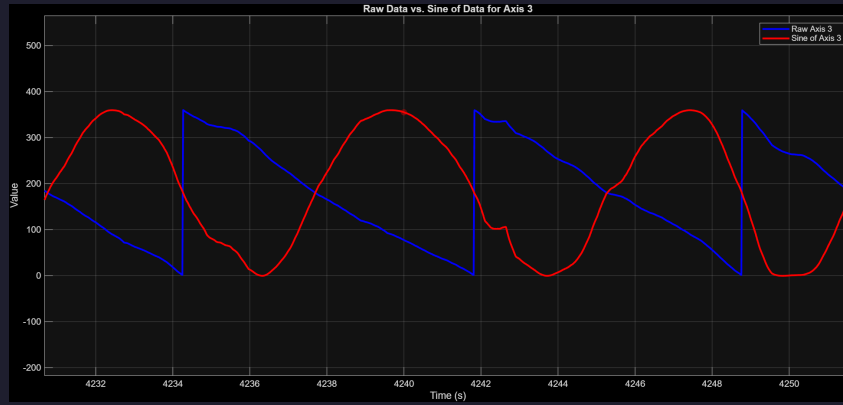
This first measurement is interesting as it shows oscillatory events in the orientation of the object. This is particularly interesting as we are looking to see variations of the movements under small environmental changes. The next step of the analysis will be to perform frequency analysis of the orientation data, to see if we can identify some dominant frequencies in the signal. This approach is relevant here because studying frequency instead of time-domain data allows to identify clear and repeatable variations of the movement under the varying conditions, without having to rely on the exact timing of the events, which are in this case heavily impacted by the experimental conditions (contact with the walls, etc...).

### Frequency Analysis :

First of all, the raw data received from the KuDAQ system is not usable as is for frequency analysis, in fact, the data is encoded in degrees, with a finite resolution, i.e. for yaw, when a full turn is performed, when the angle reaches 360, it drops back to 0 if the object continues to turn in the same direction. This creates a discontinuity in the signal, which, from our understanding and own research, can create bias in the frequency analysis, as the Fourier transform will interpret this discontinuity as a high-frequency event, which is not the case in reality.

To solve this problem, the usual solution is to perform an unwrapping of the signal before processing it. The problem here again is that the unwrapping just adds 360 when the angle drops from 360 to 0. In this way, the signal for multiple circular turns becomes a linear signal, which contains no frequency component, and is therefore not usable for frequency analysis.

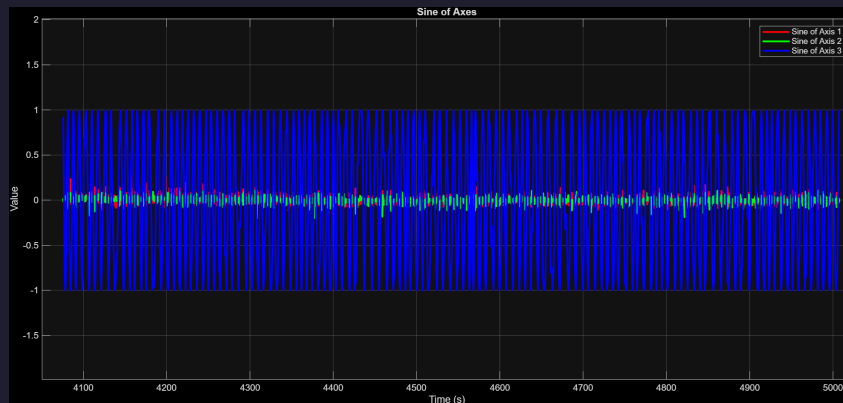
The solution we found to this problem is to perform the Fourier analysis not on the angle itself, but on the Sine of the angle. In this way, the signal becomes continuous ( $\sin(0) = \sin(360)$ ), and the frequency information is preserved in the signal, as the Sine of the angle will still oscillate with the same frequency as the angle itself, but without any discontinuity. The only drawback of this approach is that we lose some information on the amplitude of the signal, as the Sine of the angle is between -1 and 1, but for our analysis, we are mainly interested in the frequency information, so this is not a problem.



**Figure 4:** Comparison between the raw angle signal and the sine of the angle signal over time. The frequency stays the same, but the discontinuity is removed in the sine signal, which makes it more suitable for frequency analysis.

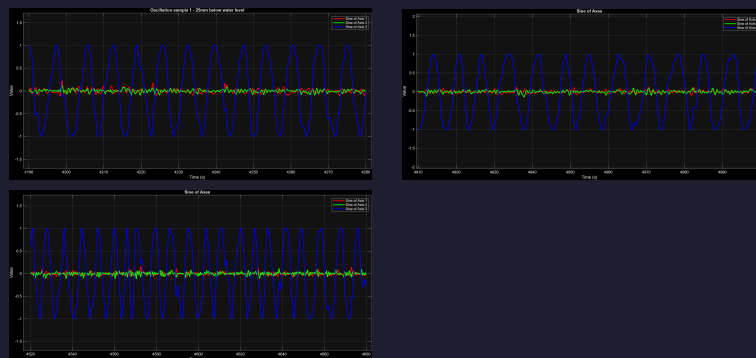
The results are the following :

First, because the 45 minutes of acquisition is a very long signal, we have the possibility to isolate pieces of signal that we found to be more qualitative, i.e. where the object is not too impacted by contact with the walls, as an example, this is the total extracted data from the 2.5 cm height acquisition, totalling 44168 data points :



**Figure 5:** Sine of the angle signal over time for the 2.5 cm height acquisition.

We decided to extract from this signal 3 pieces of data that were the most "stable", these are the pieces in question :



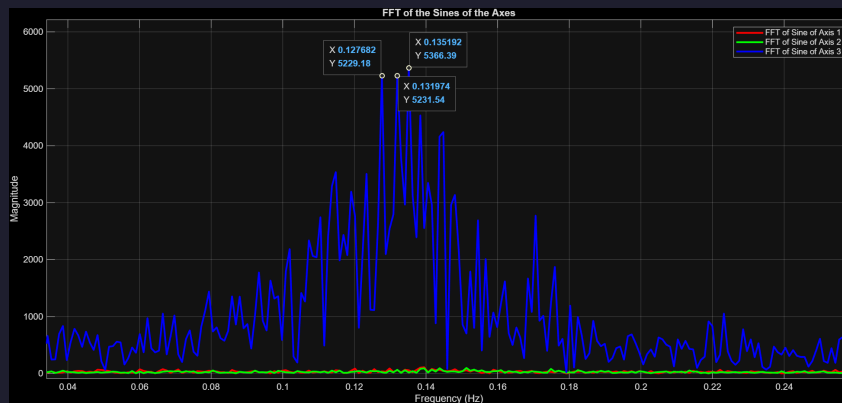
**Figure 6:** 3 extracted pieces of the sine of the angle signal over time for the 2.5 cm height acquisition.

Then, we performed a Fourier transform on these 3 pieces, using the Fast Fourier Transform (FFT) algorithm on Matlab, to extract the frequency information from the signal. The results are the following :



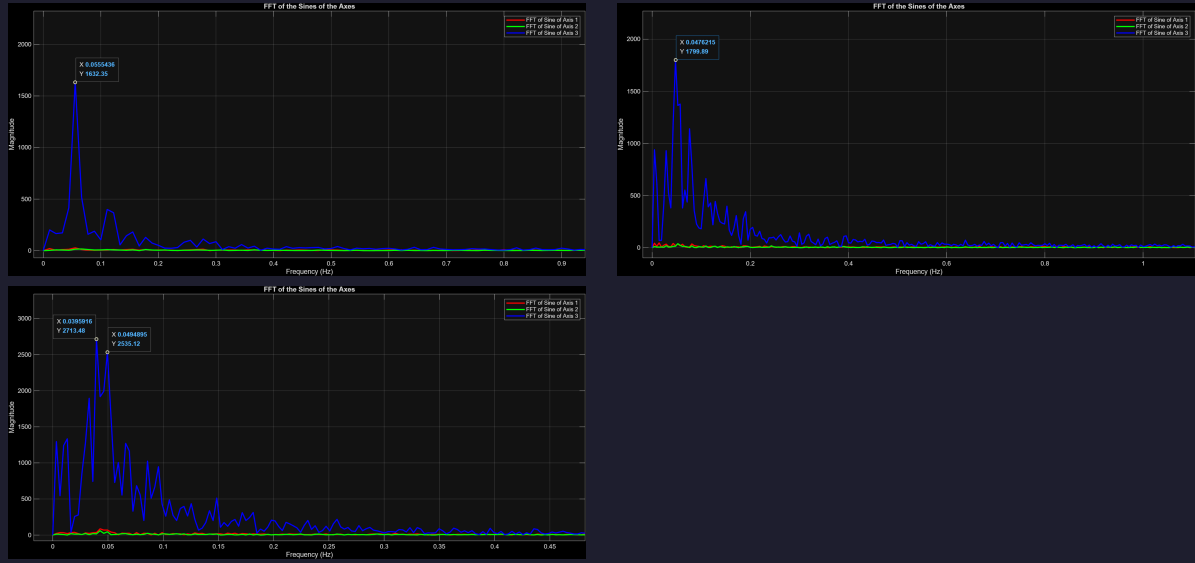
**Figure 7:** FFT of the 3 extracted pieces of the sine of the angle signal for the 2.5 cm height acquisition.

we can see that, even if the 3 pieces of signal were distant in time by several minutes, the frequency information is relatively similar, in the sense that the dominant frequency is around 0.13 / 0.14 Hz for all 3 pieces, which is coherent with the fact that the object is turning around itself at a rythm of about 1 turn every 10 seconds (1 turn every 10 seconds = 0.1 turn per second = 0.1 Hz). We can also see that there are some other frequency components in the signal, but they are not as dominant as the main frequency, and they are not consistent across the 3 pieces of signal, which makes us think that they are more likely to be noise or artefacts rather than real frequency components of the movement. We are also confident that cutting the signal in different pieces and performing separate FFT is the good method, as the FFT of the full signal turns out to have much more artifacts and many dominant frequencies compared the pieces, which can be seen on the figure below :



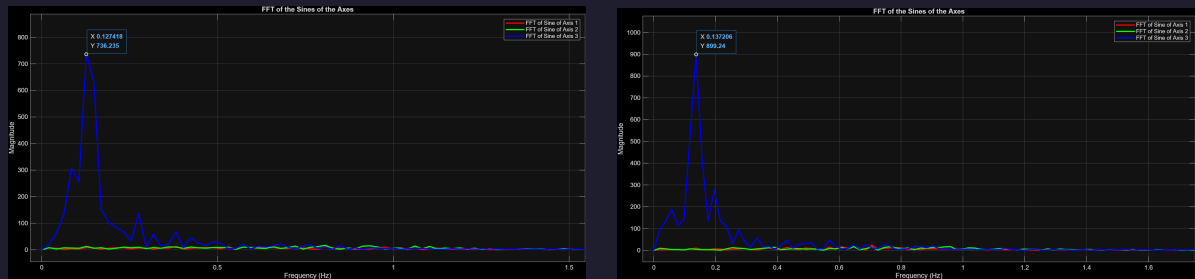
**Figure 8:** FFT of the full sine of the angle signal for the 2.5 cm height acquisition.

Using the exact same method, we performed the frequency analysis for the 5 cm height :



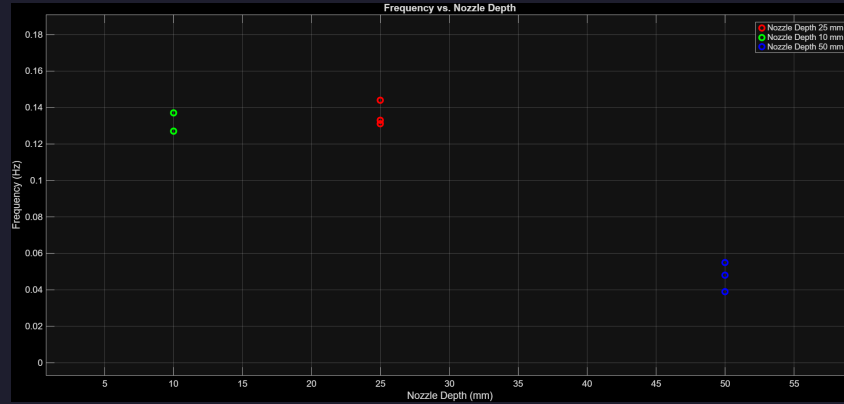
**Figure 9:** FFT of the 3 extracted pieces of the sine of the angle signal for the 5 cm height acquisition.

For the 1 cm height, we had an issue during the test as the battery of the KuDAQ ran out of power after only 15 minutes of acquisition, which is a different level of precision but we still decided to perform the exact same analysis on the data we were able to acquire :



**Figure 10:** FFT of the 2 extracted pieces of the sine of the angle signal for the 1 cm height acquisition.

In the end, we can finally plot the dominant frequency with respect to the height of the water pump in the water tank, and we can see that there is a correlation between the height of the water pump and the dominant frequency at which the object is moving, especially between the 5 cm and the 2.5 cm where the difference is more significant.



**Figure 11:** Dominant frequency of the movement of the object with respect to the height of the water pump in the water tank.

**Table 1:** Summary of dominant frequency measurements and acquisition durations.

| Nozzle Depth (mm) | Dominant Freq. (Hz) | Std. Dev. (Hz) | Duration (min) |
|-------------------|---------------------|----------------|----------------|
| 50                | 0.047               | 0.008          | 45             |
| 25                | 0.136               | 0.007          | 45             |
| 10                | 0.132               | 0.007          | 15             |

### 2.1.5 Discussion

- **Expected behavior** : The results show a clear correlation between the height of the water pump and the dominant frequency of the movement of the object, which is coherent with our initial hypothesis that the closer the water pump is to the water surface, the stronger the water flow will be, and therefore the faster the object will be moving around itself.
- **Observed deviations and anomalies** : The gap between the 2.5 cm and the 1 cm height contradicts the initial hypothesis, as the average frequency at 1 cm is slightly lower than the average frequency at 2.5 cm.
- **Potential sources of error** : The last measurement at 1 cm height was impacted by a power failure of the KuDAQ system, which caused us to only be able to acquire 15 minutes of data instead of the planned 45 minutes, which can explain the deviation observed at this height, as we had less data to analyse, and therefore less precision in the frequency analysis. In addition, the movement of the object is heavily impacted by contact with the walls of the tank, which creates a lot of noise in the signal, and can potentially create some biases in the frequency analysis.
- **Accuracy and limitations** : The battery life of the KuDAQ system was a limiting factor as it required to be charged multiple times per testing session. The mechanical conception of the leaktight box was not allowing to easily access the battery, therefore, it was chosen to perform the test until the battery runs out of power, which caused the last measurement to be impacted by a power failure.

### 2.1.6 Conclusions

This measurement was overall a success, as we were able to verify our initial hypothesis, while facing several limitations in the experimental setup. It was also a good opportunity to experiment with embedded microsystems, using sensors, wireless communication, and a complex software architecture with real-time and multi-threading features. Finally, the data analysis was also a good way of getting familiar with the FFT algorithm and the Signal Processing Toolbox in Matlab.

### 3 Additional Measurements

#### 3.1 Measurement 2: [Title to be determined]

Add content for additional measurements as needed, following the structure of Measurement 1.

## 4 Conclusion

Summarize the overall findings of the project, the effectiveness of the KuDAQ system, and recommendations for future work.



## A Raw Data

Include raw measurement data, extended tables, or additional plots here.

## B Software and Configuration

Include code listings, configuration files, or technical documentation here.